FlashDrop – Fix Admin Add Shopkeeper & Shop ID Creation System

Modify my existing FlashDrop Android application built with Kotlin and Jetpack Compose. Do not create a new application or remove any existing features.

1. Admin Panel → Add Shop

When the Main Admin opens the Admin Panel and clicks "Add Shop", display a complete Shop Registration Form matching the reference screenshot I provided.

The form must contain these fields:

1. Shop / Store Name (required)
2. Owner Full Name (required)
3. Mobile Contact Number (required)
4. Shop Category (required dropdown)
5. Physical Shop Address (required)
6. GST Number (required)
7. Create Password (required)
8. Confirm Password (required)
9. Shop Description / Key Items (optional)

Use the same clean layout as the reference screenshot, including orange icons, rounded input fields, dark borders, and the FlashDrop orange-and-green color scheme.

The Shop Category dropdown must include all supported categories, including Grocery, Printing Press, Fashion, Electronics, Food, Home & Kitchen, and other existing categories.

2. Fix the Shopkeeper ID Creation Process

The current shopkeeper ID creation process is not working. Find and fix the actual root cause instead of simply changing the UI.

When the Admin fills in the form and clicks "Create Shopkeeper Account":

- Validate all required fields.
- Validate the mobile number and GST number.
- Check that the password and confirm password match.
- Check whether the mobile number, GST number, or shop ID already exists, according to the app's existing uniqueness rules.
- Generate a unique Shop ID automatically, such as `FD-SHOP-1001`. Never assign the same ID to two shops.
- Save the shop registration details to the existing database using the project's current architecture.
- Set the account status to `APPROVED` because this account is created by an authorized Admin.
- Save the Shop ID, shop name, owner name, mobile number, GST number, category, address, and account status correctly.
- Securely store credentials using the existing authentication system. Never save plaintext passwords in the database.

- Display a loading indicator while the account is being created to prevent duplicate submissions.
- Display a success message with the generated Shop ID after the account is successfully created.
- Show a clear error message if registration fails. Do not show success unless the database confirms the operation.

3. Admin-Created Shopkeeper Login

After successful account creation, allow the Admin to view the generated Shop ID and the shopkeeper's login details.

The shopkeeper must be able to sign in through the separate Shopkeeper Sign In page using the credentials supported by the existing authentication design.

The Shopkeeper Sign In page must remain separate from the Customer Sign In page. Use the orange theme for the Shopkeeper Sign In page.

After successful authentication, verify that the account belongs to a Shopkeeper and is approved before opening the Shopkeeper Dashboard.

Do not allow a Customer account to access the Shopkeeper Dashboard just by entering a Shop ID.

4. Admin Permissions

Only an authenticated and authorized Main Admin or an existing Admin role with the required permissions can create Shopkeeper accounts through this form.

Enforce this permission in the backend or database security rules, not only by hiding the Add Shop button.

After creating an account, the Admin must be able to view the shop details, edit them, activate or deactivate the account, and access the generated Shop ID.

5. Important: Fix Existing Backend Integration

Before implementing anything, inspect the existing project and identify why Shop ID generation or account creation fails.

Check:

- Form validation and button click handlers.
- Kotlin ViewModel and repository logic.
- Firebase Authentication or whichever authentication provider the project currently uses.
- Firestore/database writes and security rules, if Firebase is used.
- Required fields, document IDs, duplicate checks, and asynchronous errors.

- Whether the generated Shop ID is actually saved and linked to the correct authenticated account.

Reuse the project's existing database and authentication provider. Do not switch providers or create a second, disconnected database.

Use a secure backend or trusted server-side transaction for unique ID generation and privileged account creation wherever the existing architecture supports it. Do not expose Admin credentials or privileged secrets inside the Android application.

6. UI and User Experience

- Match the supplied reference screenshot as closely as possible.
- Make the form vertically scrollable so all fields and action buttons remain accessible.
- Keep the header and navigation consistent with the existing FlashDrop design.
- Add clear validation messages beside invalid fields.
- Add a “Create Shopkeeper Account” primary button and a Cancel/Back option.
- Show a confirmation screen after successful account creation, including the Shop ID and account status.
- Do not break existing Customer, Shopkeeper, Both Admin, Main Admin, product, order, or delivery features.

7. Final Testing

Test the complete workflow:

1. Admin opens Add Shop.
2. The complete registration form appears.
3. Admin enters valid shop details.
4. The system creates the Shopkeeper account and generates a unique Shop ID.
5. The details are saved successfully.
6. The Admin sees the success confirmation.
7. The shopkeeper signs in through the separate Shopkeeper Sign In page.
8. The approved shopkeeper reaches the correct Shopkeeper Dashboard.
9. Invalid details, duplicate registrations, database failures, and unauthorized account creation are handled correctly.

Implement the complete working feature, not just a visual mockup. Preserve all existing functionality and report the actual cause of the original failure and the files changed.